Conversión del documento XML a código HTML (0)

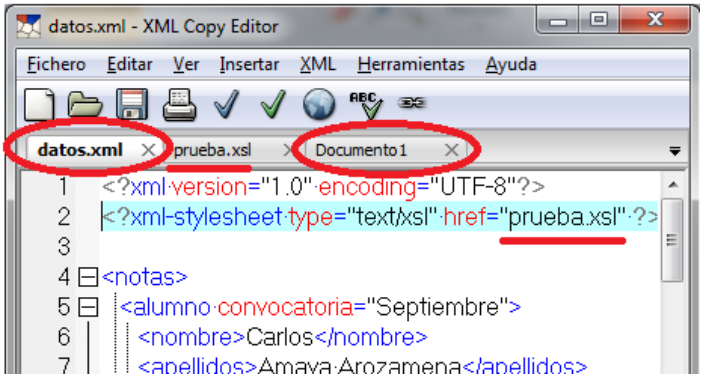
En esta unidad aprenderemos a tomar datos de un XML y mostrarlos en formato WEB. Es decir, extraeremos datos del XML y le daremos formato HTML con las instrucciones descritas en un archivo con extensión XSL.

Transformación del XML

Seguro que ya Habrás adivinado que en nuestro editor deben aparecer tanto el XML como el documento XSL.

Desde la ficha en la que se encuentra el documento XML (datos.xml) se pulsa **F8** (es la opción de XML/ XSL Trasformación…).

En ese momento si el XSL está bien formado (sin errores) se muestra el nuevo código en una nueva ficha. En el ejemplo: **DocUmento1**.



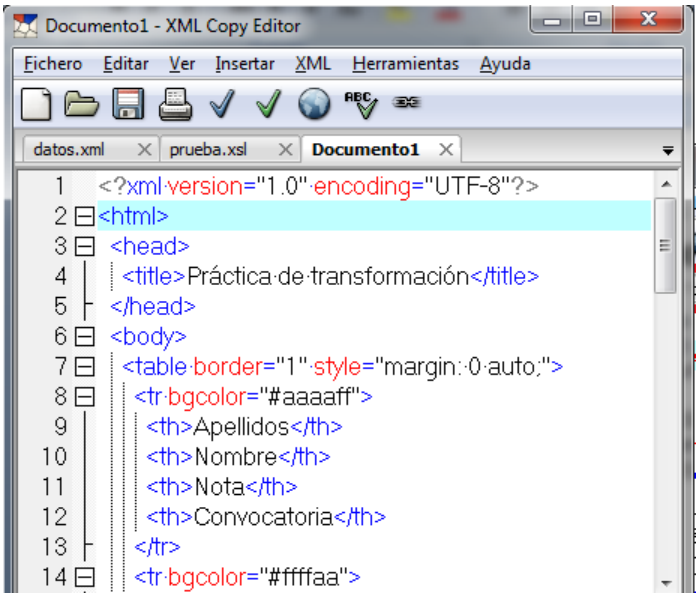
¿Qué hago con el nuevo código obtenido?

Desde esta nueva pestaña (**Documento1**) podemos hacer dos cosas:

1. Pulsar **Ctrl+B**. Es la opción de Ver/ Buscar. **Permite que el navegador por defecto interprete el código HTML**. En mi caso, Mozilla Firefox.

2. Guardar el código HTML si es que se deseara hacer esto.

IMPORTANTE. Si el documento XML lo abrís directamente con el navegador **Explorer**, éste lo interpreta y muestra el resultado de la asociación **XML-XSL**.



¿Qué hago si el xslt es correcto pero la conversión genera un error?

Este editor y procesador XSLT da problemas en ocasiones. Y más en este tipo de acciones.

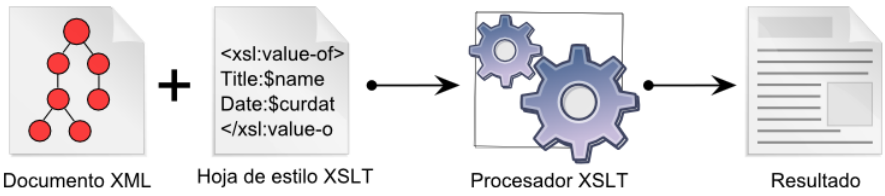
Cuando esto sucede, lo mejor que podéis hacer es:

1. Cerrar todas las fichas (xml y xsl)
2. Cerrar el propio programa
3. Volverlo a abrir
4. Y volver a cargar los dos archivos (el XML y el XSLT).

Después de esto, si todo está bien tras pulsar **F8** debe haberse creado la transformación HTML. Y Si el HTML es correcto, al pulsar **Ctrl+B** debe aparecer la **interpretación** del código en el navegador. O bien, abrís el XML con Explorer y ahí debe aparecer la interpretación.

Transformaciones XSLT (1)

Es decir, **XSLT** (eXtensible Stylesheet Language-Transformations; en español, Lenguaje de hoja de estilo ampliable para transformaciones) **describe un lenguaje, basado en XML, para transformar documentos XML** a cualquier otro formato.



Estructura de una hoja de estilo XSLT

- Una hoja de estilo XSLT es un documento XML que debe estar bien formado.
- Las hojas de estilo se guardarán siempre en archivos independientes con extensión **.xsl**

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

        . . .
</xsl:stylesheet>
```

- Entre las marcas de inicio y de fin del elemento raíz (**xsl:stylesheet**), se escribirán las **reglas de transformación** que se llamarán **plantillas.**
- **Cada regla** se definirá mediante un elemento **xsl:template** (una plantilla).
- Se utiliza el **atributo match para asociar** una plantilla con un elemento XML.
- La regla indica qué elementos del documento se van a transformar.

Enlazar documentos XML con hojas XSLT

Se puede asociar una hoja de estilo XSLT a un documento XML mediante la instrucción de procesamiento **<?xml-stylesheet ?>**. Va al principio del documento, tras la declaración XML.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="prueba.xsl" ?>
```

Documento XML de partida

Documento XML

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="prueba.xsl" ?>

<biblioteca>
        <libro>
                <titulo>Don Quijote de la Mancha</titulo>
                <autor nacimiento="1547">Miguel de Cervantes</autor>
                <publicacion fecha="1605"/>
        </libro>

        <libro>
                <titulo>La colmena</titulo>
                <autor nacimiento="1916">Camilo José Cela</autor>
                <publicacion fecha="1951"/>
        </libro>

        <libro>
                <titulo>El lazarillo de Tormes</titulo>
                <autor>Anónimo</autor>
                <publicacion fecha="1554"/>
        </libro>
</biblioteca>
```

Plantillas vacías o inexistentes

Primer ejemplo

Si no hay plantillas, el procesador simplemente recorre todos los nodos y extrae el texto contenido por cada nodo.

En el ejemplo siguiente, el resultado incluye el contenido de los nodos **<título>** y **<autor>** puesto que no hay ninguna plantilla.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Don Quijote de la Mancha Miguel de Cervantes La colmena Camilo José Cela El lazarillo de Tormes Anónimo

Segundo ejemplo

En el ejemplo siguiente, el resultado incluye el contenido de los nodos **<titulo>**, ya que no hay regla para ellos, pero los de **<autor>** se pierden porque **la plantilla es vacía**.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> **<xsl:template match="//autor"> </xsl:template>** </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Don Quijote de la Mancha La colmena El lazarillo de Tormes

Tercer ejemplo

En el caso más extremo, **si la plantilla vacía se aplica al nodo raíz**, el procesador no genera ningún resultado en el documento final ni recorre ningún nodo hijo.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> **<xsl:template match="/"> </xsl:template>** </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?>

La instrucción <xsl:value-of> (2)

La instrucción **<xsl:value-of>** extrae el contenido del nodo seleccionado.

En el ejemplo siguiente, el documento final contiene los autores de los libros porque la plantilla los genera con la instrucción <xsl:value-of>. Como se ha aplicado una plantilla al nodo **<libro>** y luego se selecciona **<autor>**, el resto de hijos (<titulo> y <publicacion>) no se recorren.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template **match="//libro"**> **<xsl:value-of select="autor"/>** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Miguel de Cervantes Camilo José Cela Anónimo

Observación. Aunque es algo que trataremos en otro capítulo (XPath), la expresión **//libro** indica que debemos posicionarnos en ese elemento, **esté en el lugar que esté**. Otra opción hubiera sido indicar toda su ruta. Es decir, /biblioteca/libro/autor. Después, al poner **"autor"**, se indica que se trata de un elemento hijo del elemento posicionado.

Modifica el código anterior y muetra los títulos de los libros con al la instrucción **<xsl:value-of>**.

En el ejemplo siguiente, se obtienen el **titulo** y el **autor** de los libros, pero **uno a continuación de otro**. **Los saltos de línea se crean tras cada aplicación de la regla**.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template **match="//libro"**> **<xsl:value-of select="titulo"/>** **<xsl:value-of select="autor"/>** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Don Quijote de la ManchaMiguel de Cervantes La colmenaCamilo José Cela El lazarillo de TormesAnónimo

No te impacientes. Poco a poco llegaremos a la transformación HTML.

De nuevo, prueba el código anterior.

Para **extraer los atributos** es preciso utilizar otra expresión (también XPath) con el carácter **"@"**.

En el ejemplo siguiente, **las fechas de publicación se obtienen gracias a la regla que extraen el valor del atributo** y **los títulos y autores se obtienen porque al no haber reglas para ese nodo se extrae el contenido**.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template **match="//publicacion"**> <xsl:value-of **select="@fecha"**/> </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Don Quijote de la Mancha Miguel de Cervantes 1605 La colmena Camilo José Cela 1951 El lazarillo de Tormes Anónimo 1554



Utilizando el código anterior realiza el siguiente cambio en la plantilla:

```
<xsl:template match="//libro">
        <xsl:value-of select="publicacion/@fecha"/>
</xsl:template>
```

¿Entiendes las expresiones XPath **match="//libro"** y **publicacion/@fecha**?
¿Y el resultado obtenido? ¿Sabes por qué ahora sólo se obtienen los **atributos**?





Para finalizar observa el siguiente código. Trata de intuir **qué** mostrará y **cómo.** ¿Hay diferencia con respecto al resultado anterior

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
version="1.0">

<xsl:template match="//libro">
        <xsl:value-of select="titulo"/>
        <xsl:value-of select="autor"/>
        <xsl:value-of select="publicacion/@fecha"/>
</xsl:template>

</xsl:stylesheet>
```

Generar texto adicional (3)



Se puede generar texto escribiéndolo en la regla; por ejemplo código html.

En el ejemplo siguiente se obtienen los **nombres** de los autores porque la regla selecciona el nodo **<libro>**, pero además generamos las etiquetas . El resultado permite intuir que **la primera transformación HTML está más cerca**.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template **match="//libro"**> ** **<xsl:value-of **select="autor"**/>** ** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Miguel de Cervantes Camilo José Cela Anónimo

Dentro de la regla podemos hacer referencia a varios subnodos. **En el siguiente ejemplo** se obtienen los nombres de los **autores** y los **títulos** de los libros porque la regla selecciona el nodo **<libro>** como en el ejemplo anterior, pero además generamos las etiquetas .

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template **match="//libro"**> ** **<xsl:value-of **select="autor"**/>** ** ** **<xsl:value-of **select="titulo"**/>** ** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Miguel de Cervantes Don Quijote de la Mancha Camilo José Cela La colmena Anónimo El lazarillo de Tormes



Ahora analiza el siguiente código. Aunque aparentemente debe realizar lo mismo que el anterior, te darás cuenta que para el mismo nodo sólo puede ejecutarse una regla. En nuestro caso, **la última**. ¡Pruébalo!

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">

<xsl:template match="//libro">
    <p><xsl:value-of select="autor"/></p>
 </xsl:template>

  <xsl:template match="//libro">
    <p><xsl:value-of select="titulo"/></p>
  </xsl:template>

</xsl:stylesheet>
```

Además de generar etiquetas, se puede generar texto. **En el ejemplo siguiente se generan frases a partir del contenido de los nodos**.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="libro"> ** <xsl:value-of select="autor"/> escribió "<xsl:value-of select="titulo"/>" ** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Miguel de Cervantes escribió "Don Quijote de la Mancha" Camilo José Cela escribió "La colmena" Anónimo escribió "El lazarillo de Tormes"

El último párrafo de la transformación no tiene sentido. Cuando practiques las estructuras de control aprenderás a filtrar los elementos para indicar cuáles se visualizarán y cuáles no.

Instrucción <xsl:text>



Este elemento inserta texto en el documento final de salida.

Nada mejor para probar esta instrucción que utilizarla para mejorar la transformación anterior. Tras el nombre del autor añadiremos: **dos puntos, carácter especial de salto de línea (
)** y comilla de apertura.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="libro"> <xsl:value-of select="autor"/> **<xsl:text> escribió:
"</xsl:text>** <xsl:value-of select="titulo"/> **<xsl:text>"</xsl:text>** </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> Miguel de Cervantes escribió: "Don Quijote de la Mancha" Camilo José Cela escribió: "La colmena" Anónimo escribió: "El lazarillo de Tormes"

Para visualizar los caracteres especiales representados por las constantes **amp; apos; gt; lt;** y **quot;** es preciso utilizar el atributo: **disable-output-escaping="yes"**. Observa el resultado.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="libro"> <xsl:value-of select="autor"/> **<xsl:text disable-output-escaping="yes">** escribió:
<</xsl:text> <xsl:value-of select="titulo"/> **<xsl:text disable-output-escaping="yes">**></xsl:text> </xsl:template> </xsl:stylesheet>	Miguel de Cervantes escribió: <Don Quijote de la Mancha> Camilo José Cela escribió: <La colmena> Anónimo escribió: <El lazarillo de Tormes>



Prueba los códigos anteriores.

Aplicar reglas a subnodos: <xsl:apply-templates> (4)



La instrucción "xsl:apply-templates" sirve para llamar a otras plantillas. Por ejemplo, desde el interior de otra.

En el ejemplo siguiente, se genera la etiqueta **<html>** además de unos párrafos con los nombres de los **libros**. Este ejemplo sí que se puede ver en el navegador ya que se interpreta como html. **¡Nuestra primera transformación de verdad XML-HTML!**



A partir de ahora se utilizará la instrucción **<xsl:strip-space elements="*" />** para mejorar el formato de presentación de los resultados, **eliminando líneas en blanco innecesarias** y **sangrando los elementos anidado**.

XSLT:	Resultado de la conversión
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:strip-space elements="*" /> <xsl:template match="/"> <html> <head> <title>Ejemplo XSLT</title> </head> <body> Libros de la biblioteca <xsl:apply-templates /> </body> </html> </xsl:template> <xsl:template match="//libro"> <xsl:value-of select="titulo" /> </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> <html> <head> <title>Ejemplo XSLT</title> </head> <body> Libros de la biblioteca Don Quijote de la Mancha La colmena El lazarillo de Tormes </body> </html>

En este código observa cómo **<xsl:apply-templates />** "llama", para que se ejecute, la plantilla que hay después del primer **<xsl:template match="/">**.



Prueba en XML Copy Editor el código anterior de la siguiente forma.

1. Desde el código XML pulsa **F8** para realizar la transformación (como siempre).
2. Desde la nueva pestaña de la transformación pulsa **Ctrl+B**, se abrirá el navegador.

Por supuesto, siempre cabe la posibilidad de guardar en un archivo el código generado.

Seguro que habrás apreciado que la instrucción **<xsl:apply-templates>** hace que **se apliquen a los subelementos las reglas que les sean aplicables**.

Pues bien, para entender mejor las posibilidades de uso de esta nueva instrucción analiza, y prueba después, el código de la siguiente página. Presta especial atención a aquello que aparece en negrita.

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> **<xsl:template match="/">** <html> <head> <title>Ejemplo XSLT</title> </head> <body> Libros de la biblioteca Títulos de libros **<xsl:apply-templates select="//titulo"/>** Autores de libros **<xsl:apply-templates select="//autor"/>** </body> </html> **</xsl:template>** **<xsl:template match="//titulo">** <xsl:value-of **select="."**/> </xsl:template> **<xsl:template match="//autor">** <xsl:value-of **select="."**/> </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> <html> <head> <title>Ejemplo XSLT</title> </head> <body> Libros de la biblioteca Títulos de libros Don Quijote de la Mancha La colmena El lazarillo de Tormes Autores de libros Miguel de Cervantes Camilo José Cela Anónimo </body> </html>

Analicemos **<xsl:apply-templates select="//titulo"/>**. Se indica que se aplique la regla al elemento hijo **titulo**. En esta plantilla aparece una nueva expresión XPath: **select="."**. Esta expresión hace referencia al contenido del nodo actual.

Algo parecido sucede con **<xsl:apply-templates select="//autor"/>** . Indica que se aplique una regla al elemento hijo **autor**.



Haz cuantas pruebas consideres sobre el código anterior.

Instrucción <xsl:attribute> (5)

La instrucción <xsl:attribute> permite generar un atributo y su valor. **Se utiliza cuando el valor del atributo se obtiene a su vez de algún nodo**.

A partir del documento XML, **se quiere generar la etiqueta <img>**. En la que el valor del atributo src será el contenido de la etiqueta **<fotografia>**.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="prueba.xsl" ?>

<libro>
        <titulo>Don Quijote de la Mancha</titulo>
        <autor>Miguel de Cervantes</autor>
        <fotografia>cervantes.png</fotografia>
</libro>
```

Primera conversión

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="/libro"> <img> <xsl:attribute name="src"> <xsl:value-of select="fotografia" /> </xsl:attribute> </img> </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?>

En la hoja de estilo XSLT, **la etiqueta <img> se escribe con apertura y cierre**, pero en el **resultado aparece como etiqueta única**.

Segunda conversión a formato HTML

XSLT:	Resultado
<?xml version="1.0" encoding="UTF-8"?> <xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0"> <xsl:template match="/"> <html> <body> <xsl:apply-templates /> </body> </html> </xsl:template> <xsl:template match="libro"> <img> <xsl:attribute name="src"> <xsl:value-of select="fotografia" /> </xsl:attribute> </img> </xsl:template> </xsl:stylesheet>	<?xml version="1.0" encoding="UTF-8"?> <html> <body> </body> </html>

Estructuras de control <xsl:for-each> (6)

A partir de ahora, utilizaremos el siguiente documento XML con el que probaremos los códigos de las distintas transformaciones:

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="prueba.xsl" ?>

<notas>
        <alumno convocatoria="Septiembre">
                <nombre>Carlos</nombre>
                <apellidos>Amaya Arozamena</apellidos>
                <matricula>m019843</matricula>
                <cuestionarios>8.0</cuestionarios>
                <tareas>8.0</tareas>
                <examen>6.0</examen>
                <final>8.0</final>
         /alumno>…

Como es de suponer, alumno se repite varias veces.
```

Además, para simplificar los códigos, en los siguientes apartados sólo se mostrarán los códigos que deben aparecer entre las etiquetas <body> … </body> del siguiente código:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:strip-space elements="*" />


<xsl:template match="/">
        <html>
                <head><title>Práctica de transformación</title></head>

                <body>

                        X

                </body>

        </html>
</xsl:template>
</xsl:stylesheet>
```

Primer ejemplo con <xsl:for-each>

A La instrucción **<xsl:for-each>** (Sintaxis: <**xsl:for-each select="expresión"**>) permite procesar de forma iterativa (repetitiva) diversos nodos del documento XML origen. Estos nodos se referencian mediante una **expresión XPath**.

XSLT	Transformación
<xsl:for-each select="/notas/alumno"> <xsl:value-of select="**nombre**"/> <xsl:text> </xsl:text> <xsl:value-of select="**apellidos**"/> **</xsl:for-each>**	<body> Carlos Amaya Arozamena José Muñoz Soto Ana Martínez de la Fuente Roberto Carrera Fernández Concepción Lalinde Priego Esther Pereda </body>

Como se puede observar, esta instrucción necesita el uso de una expresión XPath para indicar sobre qué elemento debe realizarse esa iteración o repetición del código que contiene.

HAGAMOS PRUEBAS CON EL CÓDIGO ANTERIOR

PRIMERA MODIFICACIÓN

Haz que aparezca un nuevo dato. La nota del elemento **final**.

Pero observa que la información ha de aparecer de la siguiente manera:

Apellidos, nombre - final

```
<body>
   <p>Amaya Arozamena, Carlos - 8.0</p>
   <p>Muñoz Soto, José - 8.5</p>
   <p>Martínez de la Fuente, Ana - 5.5</p>
   <p>Carrera Fernández, Roberto - 6.5</p>
   <p>Lalinde Priego, Concepción - 3.0</p>
   <p>Pereda, Esther - 2.5</p>
</body>
```

SEGUNDA MODIFICACIÓN

Si observas la tabla, ahora hay un nuevo dato, la **convocatoria** (junio o septiembre).

Separa la nota de la convocatoria con tres asteriscos.

```
<body>
   <p>Amaya Arozamena, Carlos - 8.0 *** Septiembre</p>
   <p>Muñoz Soto, José - 8.5 *** Junio</p>
   <p>Martínez de la Fuente, Ana - 5.5 *** Junio</p>
   <p>Carrera Fernández, Roberto - 6.5 *** Septiembre</p>
   <p>Lalinde Priego, Concepción - 3.0 *** Septiembre</p>
   <p>Pereda, Esther - 2.5 *** Junio</p>
</body>
```

Además, practicarás con la extracción de atributos dentro de un XML. La expresión XPath que extrae el atributo **convocatoria** (teniendo en cuenta que ya estás en el elemento alumno) es: **@convocatoria**. Hay que utilizar carácter "**@**". Es decir:**<xsl:value-of select="@convocatoria"/>**

TERCERA Y ÚLTIMA MODIFICACIÓN

En esta tercera modificación pretendo varias cosas:

1. Que aprendas a visualizar la transformación con el navegador (Pulsando **Ctrl+B**)
2. Que trabajes algunas etiquetas **HTML** que mejoren la estética en el navegador.

El Navegador ha de mostrar la tabla de la derecha.

Para que no pierdas mucho tiempo haciendo uso del ingenio buscando el código HTML más idóneo para resolver el ejercicio, observa algunas piezas del "**rompecabezas**". Seguro que sabrás colocarlas:

Apellidos	Nombre	Nota	Convocatoria
Amaya Arozamena	Carlos	8.0	Septiembre
Muñoz Soto	José	8.5	Junio
Martínez de la Fuente	Ana	5.5	Junio
Carrera Fernández	Roberto	6.5	Septiembre
Lalinde Priego	Concepción	3.0	Septiembre
Pereda	Esther	2.5	Junio

Primera "pieza"

```
<body>
        <table border="1" style="margin: 0 auto;">
                <tr> <th>Apellidos</th><th>Nombre</th><th>Nota</th><th>Convocatoria</th></tr>
```

Segunda "pieza"

```
<xsl:for-each select="/notas/alumno">
        <tr>
                <td><xsl:value-of select="apellidos"/></td>
                …
        </tr>
</xsl:for-each>
```

Estructura de control <xsl:if> (7)



Antes de comenzar el estudio de esta nueva instrucción podríamos preguntaros, **¿es posible filtrar los datos y mostrar sólo los alumnos de una determinada convocatoria?**

Nada mejor que comenzar con un ejemplo modificando la expresión XPath del **for-each**. Después, haremos lo mismo pero empleando la instrucción **xsl:if**.

XSLT	Transformación
<xsl:for-each **select="/notas/alumno[@convocatoria='Junio']"**> <xsl:value-of select="apellidos"/> <xsl:text>, </xsl:text> <xsl:value-of select="nombre"/> <xsl:text> - </xsl:text> <xsl:value-of select="final"/> <xsl:text> *** </xsl:text> <xsl:value-of select="@convocatoria"/> </xsl:for-each>	<body> Muñoz Soto, José - 8.5 *** Junio Martínez de la Fuente, Ana - 5.5 *** Junio Pereda, Esther - 2.5 *** Junio </body>



Seguro que te imaginas este primer ejercicio. Modifica el código anterior para que sólo muestre los alumnos de la convocatoria de **Septiembre**.

Primer ejemplo con <xsl:if>



La **instrucción <xsl:if>** (Sintaxis: **<xsl:if select="condición">**) permite determinar si una **acción** se llevará a cabo o no.

La **condición** es la que determina **si se ejecutarán, o no, determinadas acciones**.

Por ejemplo, en el siguiente ejemplo sólo se muestra el **nombre** y **apellidos** de aquellos alumnos que tienen en el atributo **convocatoria** el valor de **Junio**. En el ejemplo, utilizaremos esta instrucción combinándola con **<xsl:for-each>**.

XSLT	Transformación
<xsl:for-each select="/notas/alumno"> **<xsl:if test="@convocatoria='Junio'">** <xsl:value-of select="nombre"/> <xsl:text> </xsl:text> <xsl:value-of select="apellidos"/> **</xsl:if>** </xsl:for-each>	<body> José Muñoz Soto Ana Martínez de la Fuente Esther Pereda </body>



Llegados a este punto, observa los operadores **relacionales** y **lógicos** con los que debes ir familiarizándote en el uso de las condiciones XSLT y expresiones XPath.

Operadores relacionales	Operadores lógicos
= (igual) **!=** (no igual) **<** (menor que) **>** (mayor que)	**and** (conjunción) **or** (disyunción) **not**() (negación)

HAGAMOS PRUEBAS CON EL CÓDIGO ANTERIOR

PRIMER EJERCICIO

Trata de realizar la transformación que muestra la tabla de la derecha. Observa que sólo aparecen aquellos alumnos cuya nota es **cinco o superior**.

```
<body>
   <p>Amaya Arozamena, Carlos **  8.0 ** Septiembre</p>
   <p>Muñoz Soto, José **  8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana **  5.5 ** Junio</p>
   <p>Carrera Fernández, Roberto **  6.5 ** Septiembre</p>
</body>
```

Analiza la concición que has de utilizar en la instrucción: **<xsl:if test="final=5 or final>5">**

SEGUNDO EJERCICIO

Ahora, haz que sólo aparezcan los alumnos de la convocatoria de **Junio y, además, la nota es igual o superior a cinco**.

```
<body>
   <p>Muñoz Soto, José ** 8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana ** 5.5 ** Junio</p>
</body>
```

Analiza esta condición sólo si no obtienes el resultado deseado:

<xsl:if test="(final=5 or final>5) and @convocatoria='Junio'">

ÚLTIMO EJERCICIO. Ahora sin ayuda

PRIMERA PARTE.- Haz que vuelva a aparecer la información del segundo ejercicio. Pero usando la expresión del **xsl:for-each** del primer código de la hoja. **¿Sabes hacerlo?** Busca, porque en esta hoja (de dos páginas) casi, casi tienes escrita la solución.

Es decir, utilizando la instrucción:

<xsl:if test="final=5 or final>5">

Sólo deben aparecer los de la convocatoria de **Junio**.

```
<body>
   <p>Muñoz Soto, José ** 8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana ** 5.5 ** Junio</p>
</body>
```

SEGUNDA PARTE.- Si ya has conseguido el resultado anterior, para recordar un poco lo referente a tablas y poder afrontar el reto final, **retoca** la transformación para que el aspecto visual en un navegador HTML sea el que se muestra en la figura:

Apellidos	Nombre	Nota	Convocatoria
Muñoz Soto	José	8.5	Junio
Martínez de la Fuente	Ana	5.5	Junio

Estructura de control <xsl:if> (7)



Antes de comenzar el estudio de esta nueva instrucción podríamos preguntaros, **¿es posible filtrar los datos y mostrar sólo los alumnos de una determinada convocatoria?**

Nada mejor que comenzar con un ejemplo modificando la expresión XPath del **for-each**. Después, haremos lo mismo pero empleando la instrucción **xsl:if**.

XSLT	Transformación
<xsl:for-each **select="/notas/alumno[@convocatoria='Junio']"**> <xsl:value-of select="apellidos"/> <xsl:text>, </xsl:text> <xsl:value-of select="nombre"/> <xsl:text> - </xsl:text> <xsl:value-of select="final"/> <xsl:text> *** </xsl:text> <xsl:value-of select="@convocatoria"/> </xsl:for-each>	<body> Muñoz Soto, José - 8.5 *** Junio Martínez de la Fuente, Ana - 5.5 *** Junio Pereda, Esther - 2.5 *** Junio </body>



Seguro que te imaginas este primer ejercicio. Modifica el código anterior para que sólo muestre los alumnos de la convocatoria de **Septiembre**.

Primer ejemplo con <xsl:if>



La **instrucción <xsl:if>** (Sintaxis: **<xsl:if select="condición">**) permite determinar si una **acción** se llevará a cabo o no.

La **condición** es la que determina **si se ejecutarán, o no, determinadas acciones**.

Por ejemplo, en el siguiente ejemplo sólo se muestra el **nombre** y **apellidos** de aquellos alumnos que tienen en el atributo **convocatoria** el valor de **Junio**. En el ejemplo, utilizaremos esta instrucción combinándola con **<xsl:for-each>**.

XSLT	Transformación
<xsl:for-each select="/notas/alumno"> **<xsl:if test="@convocatoria='Junio'">** <xsl:value-of select="nombre"/> <xsl:text> </xsl:text> <xsl:value-of select="apellidos"/> **</xsl:if>** </xsl:for-each>	<body> José Muñoz Soto Ana Martínez de la Fuente Esther Pereda </body>



Llegados a este punto, observa los operadores **relacionales** y **lógicos** con los que debes ir familiarizándote en el uso de las condiciones XSLT y expresiones XPath.

Operadores relacionales	Operadores lógicos
= (igual) **!=** (no igual) **<** (menor que) **>** (mayor que)	**and** (conjunción) **or** (disyunción) **not**() (negación)

HAGAMOS PRUEBAS CON EL CÓDIGO ANTERIOR

PRIMER EJERCICIO

Trata de realizar la transformación que muestra la tabla de la derecha. Observa que sólo aparecen aquellos alumnos cuya nota es **cinco o superior**.

```
<body>
   <p>Amaya Arozamena, Carlos **  8.0 ** Septiembre</p>
   <p>Muñoz Soto, José **  8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana **  5.5 ** Junio</p>
   <p>Carrera Fernández, Roberto **  6.5 ** Septiembre</p>
</body>
```

Analiza la concición que has de utilizar en la instrucción: **<xsl:if test="final=5 or final>5">**

SEGUNDO EJERCICIO

Ahora, haz que sólo aparezcan los alumnos de la convocatoria de **Junio y, además, la nota es igual o superior a cinco**.

```
<body>
   <p>Muñoz Soto, José ** 8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana ** 5.5 ** Junio</p>
</body>
```

Analiza esta condición sólo si no obtienes el resultado deseado:

<xsl:if test="(final=5 or final>5) and @convocatoria='Junio'">

ÚLTIMO EJERCICIO. Ahora sin ayuda

PRIMERA PARTE.- Haz que vuelva a aparecer la información del segundo ejercicio. Pero usando la expresión del **xsl:for-each** del primer código de la hoja. **¿Sabes hacerlo?** Busca, porque en esta hoja (de dos páginas) casi, casi tienes escrita la solución.

Es decir, utilizando la instrucción:

<xsl:if test="final=5 or final>5">

Sólo deben aparecer los de la convocatoria de **Junio**.

```
<body>
   <p>Muñoz Soto, José ** 8.5 ** Junio</p>
   <p>Martínez de la Fuente, Ana ** 5.5 ** Junio</p>
</body>
```

SEGUNDA PARTE.- Si ya has conseguido el resultado anterior, para recordar un poco lo referente a tablas y poder afrontar el reto final, **retoca** la transformación para que el aspecto visual en un navegador HTML sea el que se muestra en la figura:

Apellidos	Nombre	Nota	Convocatoria
Muñoz Soto	José	8.5	Junio
Martínez de la Fuente	Ana	5.5	Junio

Estructura de control <xsl:choose> (8)



Pero qué ocurre si cuando no se cumple una condición se quiere expresar otra opción. Ésta es la principal diferencia entre las condiciones simples y las compuestas. Su sintaxis es la siguiente:

```
<xsl:choose>
        <xsl:when test="condición">
            ✘
        </xsl:when>
        <xsl:when test="condición">
            ✘
        </xsl:when>
        <xsl:when test="condición">
            ✘
        </xsl:when>
        <xsl:otherwise>
            ✘
        </xsl:otherwise>
</xsl:choose>
```

Podemos tener tantas condiciones como queramos. De forma secuencial, una tras otra, se irán comprobando sus valores. Si ninguna de ellas se cumple, se tendrá en cuenta la existencia de un elemento **<xsl:otherwise>**. Si existiese, ésa sería la acción a llevar a cabo. En caso contrario, no se llevaría a cabo ninguna acción. **Observa el siguiente código y la transformación. Además, analiza el uso de los operadores relacionales y lógicos.**

XSLT

```
<xsl:for-each select="/notas/alumno">
        <p>
        <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
        <xsl:value-of select="nombre"/><xsl:text> ** </xsl:text>
        <xsl:value-of select="final"/><xsl:text> ** </xsl:text>
        <xsl:choose>
            <xsl:when test="final&gt;=9">Sobresaliente</xsl:when>
            <xsl:when test="final&gt;=7 and final&lt;9">Notable</xsl:when>
            <xsl:when test="final&gt;=6 and final&lt;7">Bien</xsl:when>
            <xsl:when test="final&gt;=5 and final&lt;6">Suficiente</xsl:when>
            <xsl:otherwise> SUSPENSO</xsl:otherwise>
        </xsl:choose>
        </p>
</xsl:for-each>
```

Transformación

```
<body>
        <p>Amaya Arozamena,  Carlos ** 8.0 ** Notable</p>
        <p>Muñoz Soto,  José ** 8.5 ** Notable</p>
        <p>Martínez de la Fuente,  Ana ** 5.5 ** Suficiente</p>
        <p>Carrera Fernández,  Roberto ** 6.5 ** Bien</p>
        <p>Lalinde Priego,  Concepción ** 3.0 **  SUSPENSO</p>
        <p>Pereda,  Esther ** 2.5 **  SUSPENSO</p>
</body>
```

EJERCICIO RELACIONADO CON EL CÓDIGO ANTERIOR

Utilizando el **XML** con el que venimos trabajando y **modificando el código de la página anterior**, codifica un **XSLT** para que el navegador muestre el siguiente resultado:

Apellidos	Nombre	Nota	Convocatoria
Amaya Arozamena	Carlos	8.0 (Notable)	Septiembre
Muñoz Soto	José	8.5 (Notable)	Junio
Martínez de la Fuente	Ana	5.5 (Suficiente)	Junio
Carrera Fernández	Roberto	6.5 (Bien)	Septiembre
Lalinde Priego	Concepción	3.0 (SUSPENSO)	Septiembre
Pereda	Esther	2.5 (SUSPENSO)	Junio

Estructura de control: <xsl:sort>

La instrucción **<xsl:sort>** (sintaxis: **<xsl:sort select="expresión">**) es la que se utiliza para cuando queremos que la información aparezca ordenada según el valor de algún elemento.

Vamos a aprovechar el último código de la página anterior para mostrar las líneas ordenadas alfabéticamente según el valor de los apellidos. **Es suficiente con añadir una instrucción justo después del bucle for-each**. En concreto, **<xsl:sort select=”apellidos”/>**.

XSLT

```
<xsl:for-each select="/notas/alumno">
        <xsl:sort select="apellidos"/>
        <p>
        <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
        <xsl:value-of select="nombre"/><xsl:text> ** </xsl:text>
        <xsl:value-of select="final"/><xsl:text> ** </xsl:text>
        <xsl:choose>
                <xsl:when test="final&gt;=9">Sobresaliente</xsl:when>
                <xsl:when test="final&gt;=7 and final&lt;9">Notable</xsl:when>
                <xsl:when test="final&gt;=6 and final&lt;7">Bien</xsl:when>
                <xsl:when test="final&gt;=5 and final&lt;6">Suficiente</xsl:when>
                <xsl:otherwise> SUSPENSO</xsl:otherwise>
        </xsl:choose>
        </p>
</xsl:for-each>
```

Transformación

```
<body>
   <p>Amaya Arozamena,  Carlos ** 8.0 ** Notable</p>
   <p>Carrera Fernández,  Roberto ** 6.5 ** Bien</p>
   <p>Lalinde Priego,  Concepción ** 3.0 **  SUSPENSO</p>
   <p>Martínez de la Fuente,  Ana ** 5.5 ** Suficiente</p>
   <p>Muñoz Soto,  José ** 8.5 ** Notable</p>
   <p>Pereda,  Esther ** 2.5 **  SUSPENSO</p>
 </body>
```

El ejercicio final de esta hoja, seguro que te lo imaginas.

Añade la instrucción que hemos comentado (**xsl:sort**), a tu **XSLT**, para conseguir que el navegador muestre la tabla ordenada alfabéticamente según el valor del apellido. Mira la figura.

Apellidos	Nombre	Nota	Convocatoria
Amaya Arozamena	Carlos	8.0 (Notable)	Septiembre
Carrera Fernández	Roberto	6.5 (Bien)	Septiembre
Lalinde Priego	Concepción	3.0 (SUSPENSO)	Septiembre
Martínez de la Fuente	Ana	5.5 (Suficiente)	Junio
Muñoz Soto	José	8.5 (Notable)	Junio
Pereda	Esther	2.5 (SUSPENSO)	Junio

Otra versión del código anterior con llamadas a la misma plantilla (9)



Con esta hoja de trabajo sólo pretendo consolidar todos los conceptos trabajados hasta ahora.

Vamos a empezar mostrando la información usando la instrucción **<xsl:apply-templates>**, comentada en los inicios del estudio de esta nueva unidad. **Seguiremos usando el mismo XML**.

Observa en el código, sobre todo, aquello que aparece en negrita. ¡Lo entenderás!

Código XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:preserve-space elements="*"/>

<xsl:template match="/">
 <html>
    <head><title>Práctica de transformación</title></head>
    <body>
        <h1>Convocatoria de junio</h1>
        <xsl:apply-templates select="//alumno[@convocatoria='Junio']"/>
        <h1>Convocatoria de septiembre</h1>
        <xsl:apply-templates select="//alumno[@convocatoria='Septiembre']"/>
   </body>
</html>
</xsl:template>

<xsl:template match="//alumno">
            <p>
           <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
           <xsl:value-of select="nombre"/><xsl:text> ** </xsl:text>
            <xsl:value-of select="final"/><xsl:text> ** </xsl:text>
            <xsl:choose>
                    <xsl:when test="final&gt;=9">Sobresaliente</xsl:when>
                    <xsl:when test="final&gt;=7 and final&lt;9">Notable</xsl:when>
                    <xsl:when test="final&gt;=6 and final&lt;7">Bien</xsl:when>
                   <xsl:when test="final&gt;=5 and final&lt;6">Suficiente</xsl:when>
                   <xsl:otherwise> SUSPENSO</xsl:otherwise>
           </xsl:choose>
           <xsl:text>  **  </xsl:text>
           <xsl:value-of select="@convocatoria"/>
           </p>
</xsl:template>

</xsl:stylesheet>
```

Resultado de la transformación

```
<html>
  <head>
    <title>Práctica de transformación</title>
  </head>
  <body>
    <h1>Convocatoria de junio</h1>
    <p>Muñoz Soto,  José ** 8.5 ** Notable  **  Junio</p>
    <p>Martínez de la Fuente,  Ana ** 5.5 ** Suficiente  **  Junio</p>
    <p>Pereda,  Esther ** 2.5 **  SUSPENSO  **  Junio</p>
    <h1>Convocatoria de septiembre</h1>
    <p>Amaya Arozamena,  Carlos ** 8.0 ** Notable  **  Septiembre</p>
    <p>Carrera Fernández,  Roberto ** 6.5 ** Bien  **  Septiembre</p>
    <p>Lalinde Priego,  Concepción ** 3.0 **  SUSPENSO  **  Septiembre</p>
  </body>
</html>
```

XML, transformación XSLT y asociación HTML - CSS



Es posible que a lo largo del estudio de esta unidad te hayas preguntado, **¿es posible realizar una conversión XML-HTML y utilizar un CSS asociado al documento HTML?**

Seguro que ya has intuido la respuesta, **sí**. Sólo es necesario incluir en el código XSLT la etiqueta que referencia dentro del HTML el CSS. Analiza el código **XSLT**, el **CSS** y **el resultado en un navegador**.

Código XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:preserve-space elements="*"/>

<xsl:template match="/">
 <html>
<head>
<title>Práctica de transformación</title>
<link rel="stylesheet" href="estilos.css"/>
</head>
<body>
<h1>Convocatoria de junio</h1>
<table>
<tr><th>Apellidos</th><th>Nombre</th><th>Nota</th><th>Convocatoria</th></tr>
      <xsl:apply-templates select="//alumno[@convocatoria='Junio']"/>
</table>
<h1>Convocatoria de septiembre</h1>
<table>
<tr><th>Apellidos</th><th>Nombre</th><th>Nota</th><th>Convocatoria</th></tr>
      <xsl:apply-templates select="//alumno[@convocatoria='Septiembre']"/>
</table>

</body>
</html>
</xsl:template>

 <xsl:template match="//alumno">
          <tr>
              <td><xsl:value-of select="apellidos"/></td>
              <td><xsl:value-of select="nombre"/></td>
              <td><xsl:value-of select="final"/><xsl:text> (</xsl:text>
              <xsl:choose>
                  <xsl:when test="final&gt;=9">Sobresaliente</xsl:when>
                  <xsl:when test="final&gt;=7 and final&lt;9">Notable</xsl:when>
                  <xsl:when test="final&gt;=6 and final&lt;7">Bien</xsl:when>
                  <xsl:otherwise> SUSPENSO</xsl:otherwise>
              </xsl:choose>
              <xsl:text> )</xsl:text>
              </td>
              <td><xsl:value-of select="@convocatoria"/></td>
          </tr>
</xsl:template>

</xsl:stylesheet>
```

estilos.css

```
h1{
  width:900px;
  padding: 10px;
  background: #21618C;
  border: 5px solid #000000;
  border-radius: 20px;
  margin: 20px auto;
  text-align: center;
  color:#FFFFFF;
  font: bold 36px Arial, Verdana;
}

table{
  width:700px;
  border: 4px solid #000000;
  margin:20px auto;
  padding: 10px;
  background:#2E86C1;
  font: normal 18px Arial, sans-serif;
}

tr:nth-child(odd) {
  background-color:#CCDDFF;
}

tr:nth-child(even) {
  background-color:#FFFFFF;
}
```

Convocatoria de junio

Apellidos	Nombre	Nota	Convocatoria
Muñoz Soto	José	8.5 (Notable)	Junio
Martínez de la Fuente	Ana	5.5 (Suficiente)	Junio
Pereda	Esther	2.5 (SUSPENSO)	Junio

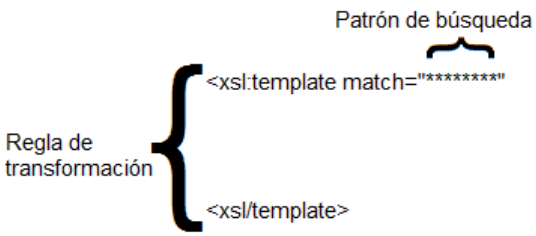
Convocatoria de septiembre

Apellidos	Nombre	Nota	Convocatoria
Amaya Arozamena	Carlos	8.0 (Notable)	Septiembre
Carrera Fernández	Roberto	6.5 (Bien)	Septiembre
Lalinde Priego	Concepción	3.0 (SUSPENSO)	Septiembre

Diferentes formas de llamar a las plantillas (10)

Las reglas de plantilla (template) son patrones para la transformación del árbol (XML).

Éstas se componen de dos partes, la primera se conoce como patrón de búsqueda y la segunda como regla de transformación.



En el siguiente código encontrarás tres formas de llamar a las plantillas:

- Mediante **apply-templates**.- Si añadimos un atributo **select** a la etiqueta **<xsl:apply-templates>** se procesará el nodo o nodos hijos que coincidan con el valor del atributo.
- Con **call-template**.- Permite llamar a una plantilla cuyo nombre (**name)** coincida con el del atributo **name** de **<xsl:call-template>**
- Y usando **call-template** con el envío de **parámetros**.- Hay que decir casi lo mismo que en el caso anterior. Pero ahora, dentro de **call-template** se define el **nombre del parámetro** y el valor **seleccionado** que se le enviará a la plantilla. En el ejemplo, observa cómo la plantilla extrae el valor asignado al parámetro, usando el símbolo "**$**".

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:preserve-space elements="*"/>

<xsl:template match="/">
    <html>
        <head><title>Práctica de transformación</title></head>
        <body>
                <h1>Convocatoria de junio</h1>
                <p>
                    <xsl:apply-templates select="//alumno[@convocatoria='Junio']"/>
                </p>

                <h1>Convocatoria de septiembre</h1>
                <p>
                    <xsl:apply-templates select="//alumno[@convocatoria='Septiembre']"/>
                </p>

                <h1>Listado de alumnos, alfabético</h1>
                <p>
                    <xsl:call-template name="orden_alfab"/>
                </p>

                <h1>Listado de alumnos, aprobados</h1>
                <p>
                    <xsl:call-template name="listado">
                        <xsl:with-param name="grupo_alumnos" select="//alumno[final&gt;=5]"/>
                    </xsl:call-template>
                </p>

                <h1>Listado de alumnos, suspensos</h1>
                <p>
                    <xsl:call-template name="listado">
                        <xsl:with-param name="grupo_alumnos" select="//alumno[final&lt;5]"/>
                    </xsl:call-template>
                </p>
        </body>
    </html>
</xsl:template>
```

```
<xsl:template match="//alumno">
        <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
        <xsl:value-of select="nombre"/><xsl:text> ** </xsl:text>
        <xsl:value-of select="final"/><xsl:text> ** </xsl:text>
        <xsl:choose>
                <xsl:when test="final&gt;=9">Sobresaliente</xsl:when>
                <xsl:when test="final&gt;=7 and final&lt;9">Notable</xsl:when>
                <xsl:when test="final&gt;=6 and final&lt;7">Bien</xsl:when>
                <xsl:when test="final&gt;=5 and final&lt;6">Suficiente</xsl:when>
                <xsl:otherwise> SUSPENSO</xsl:otherwise>
        </xsl:choose>
        <xsl:text>  **  </xsl:text>
        <xsl:value-of select="@convocatoria"/>
        <br/>
</xsl:template>

<xsl:template name="orden_alfab">
        <xsl:for-each select="//alumno">
        <xsl:sort select="apellidos"/>
        <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
        <xsl:value-of select="nombre"/><xsl:text>  **  </xsl:text>
        <xsl:value-of select="final"/><br/>
        </xsl:for-each>
</xsl:template>

<xsl:template name="listado">
        <xsl:param name="grupo_alumnos"/>
        <xsl:for-each select="$grupo_alumnos">   <!Observa el símbolo “$”  -->
        <xsl:sort select="apellidos"/>
        <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
        <xsl:value-of select="nombre"/><xsl:text>  **  </xsl:text>
        <xsl:value-of select="final"/><br/>
        </xsl:for-each>
</xsl:template>

</xsl:stylesheet>
```

Conversión HTML e interpretación del navegador (Mozilla Firefox)



```
<?xml version="1.0" encoding="UTF-8"?>
<html>
 <head>
  <title>Práctica de transformación</title>
 </head>
 <body>
  <h1>Convocatoria de junio</h1>
  <p>Muñoz Soto, José ** 8.5 ** Notable ** Junio<br/>Martínez de la Fuente,
  <h1>Convocatoria de septiembre</h1>
  <p>Amaya Arozamena, Carlos ** 8.0 ** Notable ** Septiembre<br/>Carrera
  <h1>Listado de alumnos, alfabético</h1>
  <p>Amaya Arozamena, Carlos ** 8.0<br/>Carrera Fernández, Roberto ** (
  <h1>Listado de alumnos, aprobados</h1>
  <p>Amaya Arozamena, Carlos ** 8.0<br/>Carrera Fernández, Roberto ** (
  <h1>Listado de alumnos, suspensos</h1>
  <p>Lalinde Priego, Concepción ** 3.0<br/>Pereda, Esther ** 2.5<br/></p>
 </body>
</html>
```



Convocatoria de junio

Muñoz Soto, José ** 8.5 ** Notable ** Junio
Martínez de la Fuente, Ana ** 5.5 ** Suficiente ** Junio
Pereda, Esther ** 2.5 ** SUSPENSO ** Junio

Convocatoria de septiembre

Amaya Arozamena, Carlos ** 8.0 ** Notable ** Septiembre
Carrera Fernández, Roberto ** 6.5 ** Bien ** Septiembre
Lalinde Priego, Concepción ** 3.0 ** SUSPENSO ** Septiembre

Listado de alumnos, alfabético

Amaya Arozamena, Carlos ** 8.0
Carrera Fernández, Roberto ** 6.5
Lalinde Priego, Concepción ** 3.0
Martínez de la Fuente, Ana ** 5.5
Muñoz Soto, José ** 8.5
Pereda, Esther ** 2.5

Listado de alumnos, aprobados

Amaya Arozamena, Carlos ** 8.0
Carrera Fernández, Roberto ** 6.5
Martínez de la Fuente, Ana ** 5.5
Muñoz Soto, José ** 8.5

Listado de alumnos, suspensos

Lalinde Priego, Concepción ** 3.0
Pereda, Esther ** 2.5

Objetivo: fusionar conceptos de XML, XSLT, HTML y CSS

Para poder abordar y entender qué se pretende con la tarea final de la unidad, primero, observa una última transformación del código de la hoja de trabajo número 10.



Convocatoria de junio

Apellidos	Nombre	Nota	Convocatoria
Muñoz Soto	José	8.5 (Notable)	Junio
Martínez de la Fuente	Ana	5.5 (Suficiente)	Junio
Pereda	Esther	2.5 SUSPENSO	Junio

Convocatoria de septiembre

Apellidos	Nombre	Nota	Convocatoria
Amaya Arozamena	Carlos	8.0 (Notable)	Septiembre
Carrera Fernández	Roberto	6.5 (Bien)	Septiembre
Lalinde Priego	Concepción	3.0 SUSPENSO	Septiembre

Listado de alumnos, alfabético

Nombre
Amaya Arozamena, Carlos
Carrera Fernández, Roberto
Lalinde Priego, Concepción
Martínez de la Fuente, Ana
Muñoz Soto, José
Pereda, Esther

Listado de alumnos, aprobados

Nombre	Nota
Amaya Arozamena, Carlos	8.0
Carrera Fernández, Roberto	6.5
Martínez de la Fuente, Ana	5.5
Muñoz Soto, José	8.5

Listado de alumnos, suspensos

Nombre	Nota
Lalinde Priego, Concepción	3.0
Pereda, Esther	2.5

Claro, ahora te surge una duda. **¿Y qué transformaciones se han realizado sobre el XSL de la hoja de trabajo anterior (la número 10) para que el resultado sea el que se muestra en la página anterior?** Observa dicho código.



XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:preserve-space elements="*"/>

<xsl:template match="/">
 <html>
     <head>
        <title>Práctica de transformación</title>
        <link rel="stylesheet" href="estilos.css"/>
      </head>
     <body>
        <h1>Convocatoria de junio</h1>
        <table>
            <tr><th>Apellidos</th><th>Nombre</th><th>Nota</th><th>Convocatoria</th></tr>
           <xsl:apply-templates select="//alumno[@convocatoria='Junio']"/>
        </table>

        <h1>Convocatoria de septiembre</h1>
         <table>
            <tr><th>Apellidos</th><th>Nombre</th><th>Nota</th><th>Convocatoria</th></tr>
           <xsl:apply-templates select="//alumno[@convocatoria='Septiembre']"/>
        </table>

       <h1>Listado de alumnos, alfabéticoo</h1>
         <table>
            <tr><th>Nombre</th></tr>
           <xsl:call-template name="orden_alfab"/>
        </table>

        <h1>Listado de alumnos, aprobados</h1> <!--Según valor nota, aprobados-->
        <table>
            <tr><th>Nombre</th><th>Nota</th></tr>
            <xsl:call-template name="listado">
                <xsl:with-param name="grupo_alumnos" select="//alumno[final&gt;=5]"/>
            </xsl:call-template>
        </table>

        <h1>Listado de alumnos, suspensos</h1> <!--Según valor nota, suspensos-->
       <table>
            <tr><th>Nombre</th><th>Nota</th></tr>
            <xsl:call-template name="listado">
                  <xsl:with-param name="grupo_alumnos" select="//alumno[final&lt;5]"/>
            </xsl:call-template>
        </table>

   </body>
</html>
</xsl:template>
```

```
<xsl:template match="//alumno">
        <tr>
            <td><xsl:value-of select="apellidos"/></td>
            <td><xsl:value-of select="nombre"/></td>
            <td> <xsl:value-of select="final"/>
                 <xsl:choose>
                     <xsl:when test="final&gt;=9"> (Sobresaliente)</xsl:when>
                     <xsl:when test="final&gt;=7 and final&lt;9"> (Notable)</xsl:when>
                     <xsl:when test="final&gt;=6 and final&lt;7"> (Bien)</xsl:when>
                    <xsl:when test="final&gt;=5 and final&lt;6"> (Suficiente)</xsl:when>
                     <xsl:otherwise> SUSPENSO</xsl:otherwise>
                </xsl:choose>
            </td>
            <td><xsl:value-of select="@convocatoria"/></td>
        </tr>
</xsl:template>

<xsl:template name="orden_alfab">
        <xsl:for-each select="//alumno">
             <xsl:sort select="apellidos"/>
             <tr>
                 <td>
                      <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
                      <xsl:value-of select="nombre"/>
                 </td>
             </tr>
        </xsl:for-each>
</xsl:template>

<xsl:template name="listado">
        <xsl:param name="grupo_alumnos"/>

        <xsl:for-each select="$grupo_alumnos">
             <xsl:sort select="apellidos"/>
             <tr>
                 <td>
                      <xsl:value-of select="apellidos"/><xsl:text>,  </xsl:text>
                      <xsl:value-of select="nombre"/>
                 </td>
                 <td><xsl:value-of select="final"/></td>
             </tr>
        </xsl:for-each>
</xsl:template>

</xsl:stylesheet>
```

CSS

```
body{
    background-color: #ffffdd;
}

h1{
    width:1000px;
    padding: 10px;
    background: #21618C;
    border: 5px solid #000000;
    border-radius: 20px;
    margin: 20px auto;
    text-align: center;
    color:#FFFFFF;
    font: bold 36px Arial, Verdana;
}

table{
    border: 4px solid #000000;
    margin:20px auto;
    padding: 10px;
    background:#2E86C1;
    font: normal 18px Arial, sans-
    serif;
}

tr:nth-child(odd) {
    background-color:#CCDDFF;
}

tr:nth-child(even) {
    background-color:#FFFFFF;
}
```

Descripción de la tarea final

Se pretende obtener el siguiente resultado:

Vinos de "Ribera del Guadiana"

Nombre	Tipo	Precio	Denominación de origen
Acilates 2013	Reserva	Entre 10 y menos de 20 euros	Ribera del Guadiana
Palacio Quemado Primicia	Crianza	Menos de 10 euros	Ribera del Guadiana
Palacio Quemado Reserva 2011	Reserva	Menos de 10 euros	Ribera del Guadiana
Carabal Macareno 2007 Magnum 1'5L	Crianza	Entre 20 y menos 50 euros	Ribera del Guadiana
Jaloco	Gran Reserva	Precio erróneo	Ribera del Guadiana

Vinos de "Rioja"

Nombre	Tipo	Precio	Denominación de origen
Marqués de Murrieta	Reserva	Entre 10 y menos de 20 euros	Rioja
Faustino	Crianza	Menos de 10 euros	Rioja
Castillo Ygay 1986	Gran Reserva	50 o más euros	Rioja
Barón de Ley 2011	Reserva	Menos de 10 euros	Rioja

Listado de vinos (orden alfabético)

Nombre	DO	Precio
Acilates 2013	Ribera del Guadiana	15
Alonso del Yerro 2012	Ribera del Duero	21
Barón de Ley 2011	Rioja	9
Carabal Macareno 2007 Magnum 1'5L	Ribera del Guadiana	32
Castillo Ygay 1986	Rioja	495
Corral del Obispo	Ribera del Duero	8
Faustino	Rioja	5
Jaloco	Ribera del Guadiana	***
Loes Collection	Ribera del Duero	34
Marqués de Murrieta	Rioja	16
Palacio Quemado Primicia	Ribera del Guadiana	6
Palacio Quemado Reserva 2011	Ribera del Guadiana	9
Rolland Galarreta	Ribera del Duero	16

Vinos de menos de diez euros

Nombre	DO	Precio
Barón de Ley 2011	Rioja	9
Corral del Obispo	Ribera del Duero	8
Faustino	Rioja	5
Palacio Quemado Primicia	Ribera del Guadiana	6
Palacio Quemado Reserva 2011	Ribera del Guadiana	9

Vinos entre diez y menos de veinte euros

Nombre	DO	Precio
Acilates 2013	Ribera del Guadiana	15
Marqués de Murrieta	Rioja	16
Rolland Galarreta	Ribera del Duero	16

Vinos entre veinte y menos de cincuenta euros

Nombre	DO	Precio
Alonso del Yerro 2012	Ribera del Duero	21
Carabal Macareno 2007 Magnum 1'5L	Ribera del Guadiana	32
Loes Collection	Ribera del Duero	34

Vinos de más de cincuenta euros

Nombre	DO	Precio
Castillo Ygay 1986	Rioja	495

Entonces ¿Cuál es el XML sobre el que hay que trabajar?

El de la figura. Aunque sólo se muestran tres elementos de tipo **vino** (por cuestiones de espacio), te harás una idea de la información que almacena este nodo del documento: **nombre**, **tipo** y **precio**. Además, del atributo **DO** (denominación de origen).

```
<?xml version="1.0" encoding="iso-8859-1" standalone="yes"?>
<!DOCTYPE vinos>
<?xml-stylesheet type="text/xsl" href="transform.xsl"?>

<vinos>
        <vino DO="Rioja">
                <nombre>Marqués de Murrieta</nombre>
                <tipo>Reserva</tipo>
                <precio>16</precio>
        </vino>

        <vino DO="Ribera del Duero">
                <nombre>Loes Collection</nombre>
                <tipo>Reserva</tipo>
                <precio>34</precio>
        </vino>
                •
                •
                •
        <vino DO="Ribera del Guadiana">
                <nombre>Jaloco</nombre>
                <tipo>Gran Reserva</tipo>
                <precio>***</precio>
        </vino>

</vinos>
```

Entonces…

¡Entonces nada!

- Te daré el XSL, y CSS de la segunda hoja por correo electrónico. Además del XML (alumnos).
- Lógicamente el XML nuevo, el de los vinos.
- Sólo tendrás que ir cambiando algunas cosillas para adaptarlas al nuevo árbol del XML (vinos).

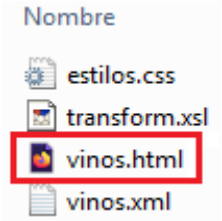
Otra información que te ayudará en la realización de esta actividad o “reto” final:

- Las dos primeras tablas (vinos de “Ribera del Guadiana y Vinos de Rioja) tienen su equivalencia con las dos primeras tablas de la transformación del XML de alumnos (convocatoria de junio y septiembre). Es decir, utilizarás la orden **<xsl:apply-templates>**, como con los alumnos.
- La tercera tabla (orden alfabético de vinos) utiliza, en la transformación, el mismo tipo de llamada a la plantilla que la tabla que muestra el orden alfabético de alumnos (**call-template)**.
- Por último, las cuatro últimas tablas en la que aparecen los vinos según el precio de los mismos, utilizan el mismo sistema de llamada a la plantilla que las dos últimas tablas de los alumnos.

  Tendrás que utilizar una llamada a la plantilla mediante el paso de un parámetro: **el que selecciona los nodos de los vinos según el precio de los mismos**. En la transformación de los alumnos, se seleccionan los nodos de alumnos aprobados (final>=5) y luego los suspensos (final<5). Ahora:

  - Por ejemplo, los vinos de menos de 10 euros se referencian, select="//vino[precio<10]"
  - Vinos de más de 10 y menos de 20 euros, select="//vino[precio>=10 and precio<20]"
  - Y bueno, dejaremos que las otras dos expresiones las descubras tú.

Consejillos sobre la nueva utilización de XML Copy Editor

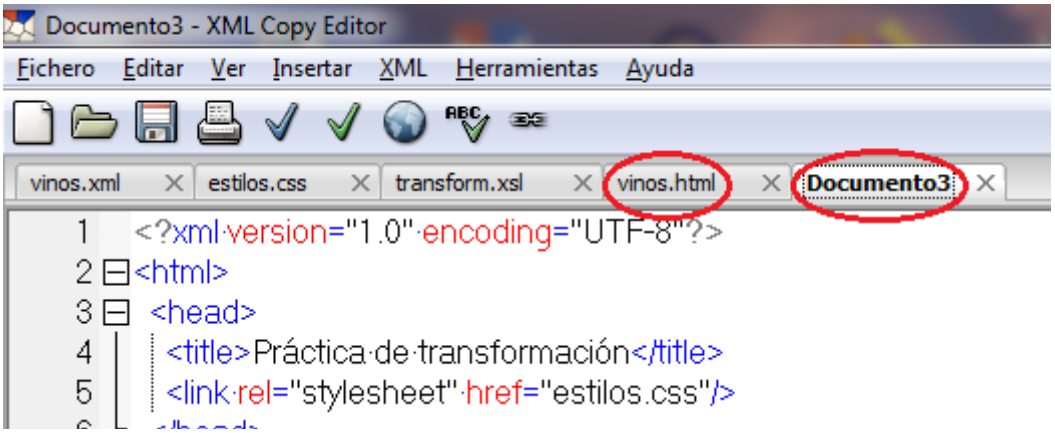


Como ahora la visualización directa de lo transformado (con **Ctrl+B**) no funcionará (ahora hay un CSS que debe ser utilizado) os recomiendo que **guardéis la transformación con la extensión html**. Después:

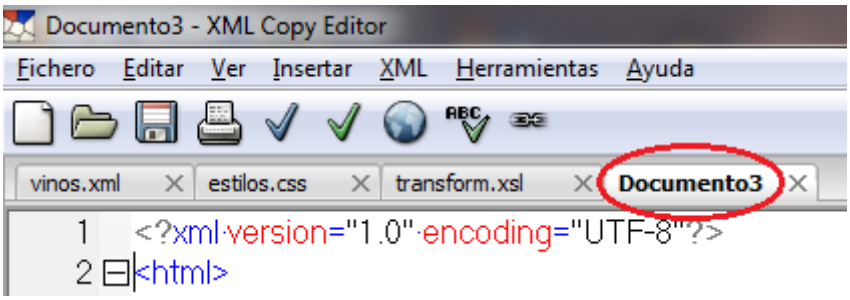
1. Podréis hacer **doble clic sobre el html** y ver el resultado.
2. **No cerréis la ventana** del navegador.
3. Así, cuando volváis a guardar la transformación con el mismo nombre, **sólo deberéis dar al icono de actualizar** (interpretará el nuevo código html).

UN PROBLEMILLA CON EL QUE OS VAIS A ENCONTRAR

Como iréis realizando y visualizando las tablas poco a poco, **es posible que alguna vez** cuando queráis guardar el resultado de la transformación, **el editor no os deje**. Es decir, que aparezca un error en la parte inferior diciendo que "vinos.html ya está abierto".Mira la siguiente imagen. **Se refiere a que hay una ficha con tal nombre**.



La solución es sencilla **elimina, primero, la ficha vinos.htm** (si es que utilizas este nombre) **para volver a guardar nuevamente el código**. Así:



Ahora **sí podrás guardar** la transformación con el nombre "vinos.html".

Y RECUERDA…

Cuando al pulsar F8 (sobre el XML) el editor lance un **mensaje** indicando que **no puede realizar la transformación** (por el motivo que sea; por ejemplo, que el error está en la línea cero), ya sabes: **cierra todas las fichas, cierra el editor, vuelve a cargar los códigos y a seguir**.

Por último, en la imagen aparece el código "**estilos.css**" no hace falta tocarlo. Lo he puesto ahí por si queréis analizarlo.

La tarea consiste en hacer sólo el XSLT.